Содержание

Power Systems Agent Benchmark: Bringing Executable Evaluation to AI Agents on Electric Power Engineering Tasks **1**
Abstract . . . 1
1. Introduction . . . 2
2. Related Work . . . 3
3. Design Requirements . . . 4
4. Architecture . . . 5
4.1 Artifact contract . . . 5
4.2 Agent-agnostic execution and a trustworthy harness . . . 5
4.3 Evaluator maturity ladder . . . 6
4.4 The held-out split as a generator . . . 6
5. Task Set . . . 6
5.1 Domain vetting of the set . . . 7
5.2 Provenance and confidence . . . 7
6. Evaluators and Metrics . . . 8
7. Reproducibility . . . 8
8. Experiments . . . 8
8.1 Public versus held-out evaluation . . . 8
8.2 Distinguishing capability from specification . . . 9
9. Limitations . . . 9
10. Conclusion . . . 10
Appendix A. Task Catalog . . . 10
Appendix B. Experiment Artifacts . . . 10

Power Systems Agent Benchmark: Bringing Executable Evaluation to AI Agents on Electric Power Engineering Tasks

Preprint draft v2 (2026-06). English version for arXiv (eess.SY; cs.AI). Word-for-word Russian companion: `paper_ru_v2.md`*. Annotated bibliography:* `bibliography.md`*; BibTeX:* `references.bib`*.*

Abstract

Executable evaluation — having an agent act and then checking the consequences of that action with a program rather than grading its prose — has become a standard way to assess tool-using AI agents in software and related settings. Electric power engineering, by contrast, has barely begun to adopt agentic AI at all: language-model use in the domain is still dominated by retrieval and text question answering, agents that act on a power system exist only as a handful of academic prototypes, and evaluation has correspondingly concerned the text a model produces rather than the consequences of an action. We bring executable evaluation to power-engineering agents. In the Power Systems Agent Benchmark an agent receives a structured task and returns a structured solution; a deterministic evaluator then recomputes the relevant engineering quantities, tests the operational constraints, and returns a feasibility flag, a normalized score, and an explicit list of violations. The benchmark spans 41 task families across eight areas of power engineering — power flow and network analysis, short circuit and protection, stability and grid-code compliance, distributed resources and storage, microgrid dispatch, reliability and restoration, power quality and standards, and forecasting — each grounded in a citable source and an engineering standard. Beyond the public set, held-out cases are not distributed as data but synthesized on demand by a per-family generator from a private seed, yielding a contamination-resistant evaluation whose

construction is open to inspection while its instances remain private. Because the launch protocol is agent-agnostic, the benchmark serves as a common platform for comparing language models and agent harnesses and for studying agent behavior on physically grounded tasks; as a reference evaluation we assess three command-line agents, which the benchmark separates cleanly and on which public and held-out performance agree. The evaluators are compact deterministic surrogates, and the task contract is designed so that their internals can later be upgraded to simulator-backed checks without changing how tasks are posed or solved.

1. Introduction

Large language models have, over a short period, moved from systems that produce text to systems that take actions. Given access to tools, a model can read structured data, call an external program, inspect the result, and decide what to do next; wrapped in such a loop, it becomes what is now commonly called an agent. This shift has been most visible in software, where agents resolve issues in real code repositories, and in business operations, where they handle customer workflows, query databases, and execute multi-step procedures through tool and API calls. The appeal is the same across these settings: an agent can carry out work that previously required a person to translate intent into a sequence of tool invocations.

It is natural to ask whether the same capability transfers to specialized engineering domains, where the work is governed not by conventions but by physical law. Electric power systems are a demanding instance. Engineering decisions there — estimating a fault current, choosing a protective relay setting, scheduling generation under security constraints, screening a network for contingencies — rest on circuit theory, on stability and protection principles, and on a dense body of standards. Whether language-model agents can perform such work is an open research question, and an active one: recent surveys document a rapid influx of agent and LLM methods into power-system analysis [llmpowersurvey2025, agenticpowersurvey2025], and early prototype systems have begun to appear [gridmind2025, xgridagent2025]. This activity is, for now, almost entirely experimental rather than deployed, which makes the question of how to measure progress a pressing one in its own right.

Measuring it is harder than it first appears, because correctness in power engineering is a physical property, not a textual one. A proposed relay setting can be eloquently justified and fail to be selective; a dispatch can be internally consistent and exceed a generator's limits; a restoration plan can read well and de-energize the wrong feeder. Errors of this kind are invisible to a human skimming the explanation and to a language model asked to judge it, because they are not errors of expression — they are violations of network physics and operating limits, and they surface only when the proposed solution is recomputed against the case. The two evaluation methods most common in agent research are, for precisely this reason, ill-suited here: question answering against a fixed reference answer cannot accommodate the many admissible solutions an engineering task allows, and grading a free-form report with a second model [zheng2023judging] rewards a fluent rationale even when the underlying decision is unsound — a tendency reinforced by the documented biases of such judges [wang2023fairevaluators].

In software and tool-use settings this difficulty has been met by executable evaluation: an agent's output is not scored as prose but applied and checked by a program. SWE-bench [jimenez2024swebench] runs a repository's own test suite against the agent's patch; τ-bench [yao2024taubench] compares the final state of a database against an annotated goal; terminal-bench [terminalbench2025] inspects the effect of an agent's commands on a live shell. The principle these share — judge the outcome of an action, not its description — is exactly what power-engineering evaluation needs, and it has not been applied to the domain at any breadth.

This paper does so. We introduce the Power Systems Agent Benchmark, in which each task supplies an agent with structured inputs and a required output schema; the agent returns a structured solution; and a deterministic evaluator recomputes the relevant engineering quantities, tests the hard constraints, and returns a feasibility flag, a normalized score in the unit interval, and an explicit list of violations. The agent's natural-language explanation is recorded but excluded from scoring. The public portion comprises 41 task families across eight areas of power engineering, each grounded in a citable source and an engineering standard. The held-out portion is not distributed

as data: a generator produces held-out cases on demand from a private seed, so that the construction of the held-out set is open to inspection while its instances remain unknown to those being evaluated.

This benchmark is our primary contribution, and it is best understood as three things at once. It is, first, a broad executable test of power-engineering competence: 41 task families spanning eight areas of the field, each posed through a uniform contract and checked by a deterministic evaluator that recomputes the underlying physics, so that an agent is measured by what its decisions do rather than by how it describes them. It is, second, a contamination-resistant evaluation: rather than ship a fixed held-out set that ages and leaks, we provide a per-family generator that synthesizes physically consistent, solvability-checked cases from a private seed, keeping the held-out construction auditable while its instances stay private. And it is, third, an agent-agnostic platform: because participating reduces to reading a task file and writing a solution file, the same benchmark can compare different language models and different agent harnesses under one protocol, and can serve as a substrate for studying agent behavior on physically grounded tasks. We accompany the benchmark with a reference evaluation of three command-line agents and, in the course of running it, report a methodological observation — that a failure shared by all agents is a reliable signal of a defective task or evaluator — useful enough to recommend in its own right (Section 8.2).

2. Related Work

Our work draws on three lines of research, and is most easily located by where it follows each and where it departs.

The first is executable evaluation of agents. SWE-bench [jimenez2024swebench] and the earlier pass@k construction of Codex/HumanEval [chen2021codex] established that an agent's output can be judged by executing it; τ-bench [yao2024taubench] generalized this to tool-and-user interaction by comparing end states and proposed pass^k as a reliability measure; terminal-bench [terminalbench2025] and MLE-bench [chan2024mlebench] extend the idea to terminal work and machine-learning engineering, while broad suites such as AgentBench [liu2024agentbench], GAIA [mialon2023gaia], and WebArena [zhou2024webarena] probe general tool use. We adopt this paradigm; our point of difference is coverage, since none of these targets power-system engineering and its physical constraints.

The second line concerns *how* to score, and it is the reason we recompute rather than judge. What ultimately matters is the effect of an agent's actions, not the fluency of its account of them, and for verifying effects the strongest available tool is a deterministic check. This is not a new idea — it is the basis of software testing, and of competitive programming, where a submission is accepted only if it passes a fixed suite of tests — and wherever a task admits such a check, as every task in this benchmark does, it should be preferred: it is exact, reproducible, and immune to the biases that attend a language-model judge, the biases of position, length, and self-preference [zheng2023judging, wang2023fairevaluators] that are most damaging exactly when a wrong answer is fluent. Model-based judging retains a role where no deterministic check is feasible, and recent work extends it from single responses to whole agent trajectories under the heading of *agent-as-a-judge* [zhuge2024agentjudge, aisjudgeais2025]. We expect that as the benchmark grows to include scenarios too open-ended for a closed-form evaluator, judging of this kind — applied to actions and trajectories, not to prose — will become appropriate, and we leave that path open; for the tasks presented here, deterministic recomputation suffices, and the agent's rationale is kept only as a diagnostic.

The third line is domain-specific. Surveys [llmpowersurvey2025, agenticpowersurvey2025] record a rapid growth of LLM and agent methods in power systems with no settled means of evaluation. The closest effort to ours is PowerAgentBench [poweragentbench], part of the actively developed PowerAgent ecosystem at Harvard SEAS [zhang2025poweragent]. We adopt the same core design it uses — a public/held-out split with an evaluator that recomputes physical validity — which we do not claim as novel: PowerAgentBench is a prior instance of it in this domain, and contamination-resistant held-out evaluation is by now a broadly established idea

[contaminationsurvey2025, white2024livebench]. ElecBench [zhou2025elecbench] scores dispatch scenarios largely through text and judges, and PFBench [pfbench2025] applies programmatic checking but only to power flow. The prototype agents now appearing — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026], PFAgent [pfagent2026] — are the intended subjects of a benchmark of this kind. What distinguishes our work is not the split but its emphasis: breadth across many task families, held-out cases generated from a seed rather than stored, and an explicit quality-control procedure.

Two further bodies of work inform specific choices. Reinforcement-learning grid environments, principally Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective], provide a mature setting for sequential topology control and are built on established power-system simulators — pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa] — refined over many years to compute power-system physics. We address the different problem of single-shot engineering decisions across many subtasks under an agent-agnostic contract, and our current evaluators do not use these simulators at all: each is a compact, self-contained deterministic check. We name them only as the natural backend for higher-fidelity future versions of the evaluators, not as a present dependency. Finally, the literature on benchmark integrity informs the held-out split: because training-data contamination inflates results [contaminationsurvey2025, statictodynamic2025], dynamic and refreshed benchmarks have become the standard response [kiela2021dynabench, white2024livebench, jain2024livecodebench], and a seed-driven generator is a prevention mechanism of the same family. The harness follows the prescriptions of [kapoor2024agentsmatter] on cost, reproducibility, and the separation of failure classes, and respects the limits of best-of-N characterized in [brown2024monkeys].

3. Design Requirements

The system is shaped by a small set of requirements, stated here at the outset because most of what follows is a consequence of them; we note for each where it is realized.

1. **Executable, physical verification.** A solution must be checked by running code rather than by matching strings, and that code must recompute the relevant physical quantities and test the operating constraints, not compare against stored text. The remaining requirements largely follow from this one (the contract is given in Section 4.1, the evaluators in Section 6).
2. **A leak-proof public/held-out split.** Public cases must support development and held-out cases a fair comparison; crucially, held-out answers must be excluded by construction rather than by policy, so that no amount of scrutiny of the released material reveals them (Section 4.4).
3. **Agent agnosticism.** Since the point is to compare models and harnesses on equal terms, launching an agent must not depend on any particular one — participation reduces to reading a task file and writing a solution file (Section 4.2).
4. **Provenance and machine-readable results.** Every task must declare where it comes from, with a confidence level (Section 5.2), and every result must be machine-readable — a feasibility flag, a score, and an explicit list of violations (Sections 4.1 and 6) — so that runs can be aggregated and audited without anyone reading prose.
5. **Room to grow.** The benchmark must be able to become more demanding without breaking compatibility, so that replacing a closed-form check with a simulator-backed one changes an evaluator's internals while leaving the task and solution formats untouched (Section 4.3).
6. **Specification-quality control.** Finally, because a benchmark's own tasks and evaluators can themselves be wrong, it must provide a means of telling a defect in its specifications apart from a genuine difference in agent capability (demonstrated in Section 8.2).

4. Architecture

4.1 Artifact contract

The benchmark is organized around a single linear contract (Figure 1): a task description is given to an agent, the agent produces a structured solution, an evaluator consumes the solution, and an evaluation result is emitted. A JSON schema fixes each of the three artifacts. A task carries its identity and family, its public inputs, the schema its solution must follow, the hard constraints that will be checked, the metrics, the provenance, and a bilingual presentation block. A solution carries the agent's structured answer and an optional rationale. An evaluation result carries the feasibility flag, the score, the list of violations, and any supporting evidence. Because the formats are fixed independently of how a task is solved or checked, an agent and an evaluator need share nothing beyond these schemas.



Рис. 1: Figure 1: The benchmark contract. A task is posed to any command-line agent, which writes a structured solution; a deterministic evaluator recomputes the result and emits a machine-readable verdict.

A short-circuit task, abbreviated, illustrates the form:

```
{
  "family": "three_phase_short_circuit",
  "public_inputs": {"short_circuit_case": {
    "base_mva": 100.0, "voltage_kv": 10.5,
    "voltage_factor_cmax": 1.1, "voltage_factor_cmin": 1.0,
    "positive_sequence_impedances_pu": [ /* ... */ ],
    "fault_resistance_pu": 0.005}},
  "submission_schema": {
    "ik_max_ka": "max initial symmetrical current (kA)",
    "ik_min_ka": "min current with fault resistance (kA)", "...": "..."},
  "hard_constraints": [
    "maximum current uses c_max over a bolted fault",
    "values match the independent positive-sequence calculation"],
  "metrics": [{"name": "feasible", "direction": "boolean", "...": "..."},
              {"name": "score", "direction": "maximize", "...": "..."}],
  "source_provenance": [{"title": "IEC 60909 short-circuit calculation",
                         "confidence": "high", "...": "..."}]
}
```

4.2 Agent-agnostic execution and a trustworthy harness

For a comparison across agents to be meaningful, the apparatus that runs them must satisfy three properties beyond merely invoking each agent on each task. The first is isolation: an agent must not be able to read the evaluator or any stored answer. Each task is therefore packaged with its answer key and evaluator internals removed, and is executed in a sandbox that has no path back to the benchmark's own code. This is not a precaution in the abstract. An agent granted autonomous filesystem access will, if the evaluator is reachable, consult it: in the course of this study one agent located the checker for a task and graded itself against it, a behavior whose only barrier had been a sentence of instruction. Instruction-level prohibition is not isolation for such an agent, and genuine isolation must be structural.

The second property is robustness to malformed output. An agent may return a solution that is syntactically valid but meaningless — null values, text where a number is required, the wrong structure entirely — and the evaluator must treat this as an ordinary failed solution, returning infeasible with a list of the offending fields, rather than raising an exception that halts the entire run. Without this property a single malformed solution can abort a multi-task evaluation partway and leave an incomplete, incomparable record.

The third property is a principled separation of failure classes. A run can fail for reasons that have nothing to do with the agent's competence — a transient provider error, a dropped connection, an exhausted usage quota — and conflating these with genuine failures contaminates the measurement. The harness classifies each failure and retries only those attributable to infrastructure, never those in which the agent produced a poor or absent solution within a healthy environment; retrying the latter would silently turn the benchmark into a best-of-N search and inflate every score. Each result records how many attempts it took and why earlier attempts failed, and a run reports its metrics both with and without infrastructure retries, so that provider reliability is never mistaken for agent capability.

4.3 Evaluator maturity ladder

The evaluators are allowed to mature in place along a ladder: from a closed-form or graph-based surrogate, through a deterministic engineering model, to a check backed by a full power-system simulator, and ultimately to a stochastic or large-scale scenario suite. The evaluators presented here occupy the first two rungs. The contract of Section 4.1 is what makes the ladder usable: because the task and solution formats are fixed independently of how a check is computed, the internal step of replacing, say, a linearized power-flow surrogate with a full AC solution upgrades an evaluator without altering anything an agent sees.

4.4 The held-out split as a generator

The benchmark does not store its held-out cases at all. In place of held-out data files, it provides a generator: a collection of deterministic, physics-aware perturbations, one per family. A held-out case is produced from a public case by redrawing its numerical content — network parameters, weather series, load profiles, a fault location together with the indicator readings and customer complaints it implies — under a private random seed, and a draw is accepted only if the family's reference solver still solves it and the evaluator confirms the solution feasible, with a deterministic redraw otherwise. Whoever operates a leaderboard keeps the seed private; anyone else can read exactly how held-out cases are constructed without learning the particular instances of any round. This makes the held-out set, normally the most leak-prone component of a benchmark, auditable in mechanism and private in content at the same time.

5. Task Set

The public benchmark comprises 41 task families, grouped into eight areas of power engineering to make their coverage legible. Network analysis and operating constraints include bus-admittance-matrix construction, corrective reactive-power dispatch for voltage limits, contingency screening, post-contingency corrective switching, and dynamic line rating. Short circuit and protection include three-phase and single-line-to-ground fault calculation, breaker and relay setting from fault levels, distance-protection zone setting, overcurrent coordination, and cable sizing. Stability, grid code, and inverter-based resources include critical clearing time by the equal-area criterion, transient-stability classification, fault-ride-through compliance, current-limited inverter fault response, and minimum synchronous-generation share under inertia and short-circuit constraints. Distributed resources, photovoltaics, electric vehicles, and storage include Volt-VAR control, vehicle-to-grid scheduling and voltage support, battery ancillary-service response, and a soft-cost levelized-cost calculation. Microgrids and dispatch include economic and rolling-horizon dispatch, islanded active/reactive dispatch, robust dispatch under uncertainty, hydro-thermal unit commitment, and single-contingency-secure

generation commitment. Reliability and restoration include fault isolation and service restoration, faulted-section localization, fault-indicator placement, reliability-index improvement, and operator switching and shedding. Power quality, standards, assets, and cybersecurity include voltage-compliance checking to a supply standard, power-quality event classification, harmonic-limit compliance, transformer thermal loading and insulation ageing, bad-data identification in state estimation, secured-measurement placement, and lead-station handover in a multi-agent frequency scheme. Forecasting under uncertainty includes wind power forecasting and prediction-interval estimation. Every family carries a documented source and a confidence level, and the appendix lists the full catalog with identifiers.

5.1 Domain vetting of the set

Assembling tasks from the literature is not the same as making them correct, and a domain review of the initial set removed four classes of factual defect. Some were errors of physics: in one task a rate-of-change-of-frequency expression placed system frequency in the denominator rather than the numerator, a squared-frequency error that left the inertia constraint it was meant to enforce permanently slack; restoring the swing-equation form and rebalancing the case corrects it. Some were conceptual: a task nominally about false-data-injection detection had included the attack labels among its inputs and so reduced to a threshold comparison, and it was reworked into a genuine weighted-least-squares state estimation with an iterative largest-normalized-residual test — a setting in which residual smearing defeats the naive test — with stealthy injection scoped out explicitly. Some were degenerate: a critical-clearing-time task read its answer from precomputed stability flags, and was replaced by an analytic equal-area calculation. And some overstated a standard: a short-circuit task invoking a fault-calculation standard had omitted the standard's voltage-correction factor and folded arc resistance into the maximum current, understating that current by roughly ten percent in the non-conservative direction, and was reworked into the standard's own maximum/minimum current pair. Finally, three families that on inspection lay outside power engineering — two concerning building heating and one a generic variance decomposition — were retired in favor of previously uncovered domains: transformer thermal loading, cable sizing, and harmonic-limit compliance.

5.2 Provenance and confidence

Because the task set is drawn from the engineering literature rather than authored from scratch, each family records where its formulation comes from and how firmly it is tied to that origin. Every task carries a provenance entry — a source title, a locator (a digital object identifier or URL for an article, a clause for a standard, or a full citation for a textbook), and a short note on how the source maps onto the executable case — together with a stated confidence level. The confidence level is a deliberate, declared judgment rather than a measured quantity, and we assign it by a fixed rubric so that it is reproducible:

- **high** — the task reproduces a method defined in a published standard or a canonical textbook, cited specifically, and the executable case follows that method directly (for example, the short-circuit, transformer-loading, and harmonic-compliance tasks follow named international standards, and the critical-clearing-time and bad-data tasks follow standard textbook procedures);
- **medium** — the task is tied to an identified primary source, or to a universally taught engineering method written out as its governing equations, and the case is a faithful compact instance of it;
- **low** — no single primary document is pinned: the method is standard but the task is an adaptation in its spirit rather than a reproduction of a specific source.

Under this rubric, of the 41 families 11 are rated high, 28 medium, and 2 low; the two low cases (uncertainty-aware dispatch and minimum synchronous share) keep that rating because no single source we could verify fully grounds their exact formulation, and we prefer to say so than to inflate the label. We regard the scheme as a floor on transparency rather than a guarantee of fidelity: it lets a reader trace any task to its origin and see at a glance how tightly it is bound there, and it makes explicit which tasks are faithful renderings of a standard and which are looser adaptations of common practice.

6. Evaluators and Metrics

What an evaluator returns is uniform across the benchmark — a feasibility flag, a score in the unit interval, a list of violations stated in words, and task-specific evidence — but how it arrives at a score depends on the kind of task. An exact computation is scored against numeric tolerances. A control or optimization task is scored by combining feasibility on the constraints with quality on the objective; where the optimum is not unique, as in dispatch, the evaluator re-simulates the submitted decision and scores the gap to the optimal cost, so that an alternative optimum of equal cost receives full credit and a feasible but more expensive solution receives partial credit. A classification or localization task is scored against exact sets with enumerated misses, and a standards-compliance task as pass/fail with diagnostic fields. At the level of a whole run we report two aggregates that need not agree: the share of tasks meeting the hard constraints, and the share scoring exactly one. Because these can rank agents differently, we recommend reporting both alongside the mean score.

7. Reproducibility

The benchmark is intended to be re-run rather than taken on trust. A single command runs the unit tests over the evaluators, the reference solvers, the harness, and the generator; validates every artifact; evaluates each of the 41 public sample solutions individually; and runs the reference solvers across the public set. Those solvers are built on the same computational kernels as the evaluators and return all 41 cases feasible at a mean score of 0.997. We present this figure as a self-consistency check on the contract rather than as evidence that the tasks are easy — a distinction that proves important, because, as Section 8.2 shows, this very self-consistency is blind to errors in an evaluator: a reference solver derived from a faulty evaluator agrees with it by construction. Continuous integration runs the full check on every change.

8. Experiments

We evaluated three command-line agents under identical prompts, a fixed per-task timeout, and the infrastructure-retry policy of Section 4.2, with no adaptation of the agents to the benchmark: a coding agent on a frontier general model at medium reasoning effort, an IDE agent on a proprietary model, and an open-source agent on a small open model. We refer to them below as codex, cursor, and opencode, with the exact versions recorded in the appendix.

8.1 Public versus held-out evaluation

The reference round was run after the corrections described in this paper. Each agent attempted the public case of every family and one freshly generated held-out case per family drawn from a new private seed; no run encountered an unresolved infrastructure failure, so the metrics with and without retries coincide.

agent	public feasible	public perfect	public mean	held-out feasible	held-out perfect	held-out mean
codex	40/41	36/41	0.971	40/41	37/41	0.969
cursor	39/41	35/41	0.947	40/41	36/41	0.969
opencode	37/41	35/41	0.900	38/41	35/41	0.920

Two observations follow. The benchmark discriminates between agents: codex and cursor, both near a mean of 0.97, sit clearly above opencode at 0.90 to 0.92, and do so on both splits. And, on corrected specifications, the public and held-out splits are comparable — each agent's held-out score lands within noise of its public one. We report this plainly because an earlier round, run before the specification corrections, had appeared to show a sharp divergence in which the public leader collapsed on held-out cases. That divergence proved to be an artifact

of the defects discussed next, which happened to fall hardest on one agent; it vanished once the defects were corrected. We therefore make no claim of an intrinsic difficulty gap between public and held-out cases. The case for a held-out split is instead prospective: a published set ages as models train on it, and a generator is what keeps an uncontaminated draw available. As mild corroboration, the agents do not score higher on the published public set than on freshly generated held-out cases — the signature one would expect from memorization is absent at this case size.

8.2 Distinguishing capability from specification

Reading the failures across agents rather than per agent proved the most useful analysis available, because the failures separate into two kinds. When agents fail a task in different ways — one breaching a voltage limit where the others hold it, one returning a wrong ride-through verdict, one inverting a sign — the disagreement is signal, and it is what a leaderboard should reward. But when every agent fails a task in the same way, the more likely explanation is that the task, not the agents, is at fault. Treating that pattern as a diagnostic surfaced ten defects in the specifications: a field whose description contradicted its name, so that two agents correctly computed a fraction where the evaluator expected a percentage; nested keys that did not match the checker; a semantically empty zero entry that was nonetheless rejected; a demand for an exact match to one of several optima of equal cost; unstated conventions for a power curve, for quantiles, and for the terms of a heat balance; a hardcoded dispatch policy; and an underspecified scoring formula. Each correction was confirmed by re-evaluating the affected family and observing the agents recover. We propose the underlying rule — a unanimous failure should be suspected as a defect in the specification or the evaluator — as a routine quality-control step for executable benchmarks.

The most consequential instance pointed not at a task but at an evaluator, and is worth stating in full, as it is the clearest evidence we have for the value of the method. On one held-out reliability case concerning faulted-section localization, all three agents identified the same faulted section, a conclusion consistent both with the fault-current-indicator readings and with the pattern of customer complaints; the evaluator demanded a different section that in fact contradicted the case's own indicator readings. The agents were correct and the evaluator was wrong. The cause was an off-by-one error in the evaluator's indicator model: it had placed the faulted section's downstream node on the fault-current path, so that an indicator at that node was predicted to register a fault current it would not in fact see, whereas an indicator registers fault current only when the fault lies strictly downstream of it. What makes this more than a corrected bug is that the benchmark's own safeguards could not have caught it. The reference solver used for self-checking is derived from the same function as the evaluator, so solver and evaluator agreed with each other and the public sample passed; the error was invisible from within and became visible only because three independent agents brought the correct physics to bear. An executable benchmark, in other words, can harbor a latent domain error in the very code that defines correctness, and a matrix of agents is a means of finding it — a caution we would extend to any benchmark whose reference answers are derived from its evaluator.

9. Limitations

Several limitations bound what these results support. The public cases are compact — networks of a few buses and short time series — so they test whether an agent commands a method, not whether it scales to industrial data volumes. The evaluators are deterministic surrogates, and no claim of industrial AC accuracy is warranted until simulator-backed checkers are in place. With one case per family per split, a ranking over 41 points is statistically fragile, though the generator makes enlarging the sample inexpensive. The reference solver's perfect public score is a self-consistency check on the contract, not a lower bound on difficulty, and cannot validate the evaluator. The seed of a held-out round is spent once the round is disclosed, so a citable leaderboard requires a fresh seed on an isolated host. The sandbox isolates by relocation rather than containerization, which removes leakage through relative paths and environment but not a determined search for the evaluator by absolute path, so competitive

use would want host-level containers. We run one configuration per agent and so do not estimate the variance of model sampling. Finally, evaluating an agent built on the same model family as the benchmark's authors, on an account whose budget is shared with development, proved infeasible because it exhausted that budget; a clean self-evaluation would require a dedicated account and an explicit conflict-of-interest statement.

10. Conclusion

We have brought executable evaluation, by now standard for software and tool-use agents, to electric power engineering, where assessment had remained textual. The benchmark turns power-system tasks into checks that recompute physics, and on a matrix of current agents it discriminates between them consistently across its public and held-out splits. Its most transferable contribution is not any single task but a method: reading a unanimous failure as a defect signal, which over the course of this work uncovered ten specification defects and a latent error in an evaluator's physics that the benchmark could not have caught by self-consistency alone — a reminder that a reference answer derived from an evaluator cannot certify it. The lasting value of the artifact lies in the contract, the domain-grounded task set, the evaluators, the seed-driven held-out generator with its prospective resistance to contamination, the isolated harness, and the reproducible quality-control procedure; the work ahead is to fill the evaluators with simulator-backed implementations without disturbing the contract that holds them together.

Appendix A. Task Catalog

The full catalog of 41 families, in English and Russian, with identifiers, is provided with the benchmark.

Appendix B. Experiment Artifacts

The per-agent, per-split summaries and logs of the reference round, the earlier development rounds, and the record of specification corrections are provided with the benchmark.